

澳門大學
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU



信息學院
Faculdade de Ciência da Informação e Computação
Faculty of Information Science and Computing

Convolution

EMEN 7029 & EMEN 7104 - Special Topic In Electromechanical Engineering / Robotics

Qingbiao Li

University of Macau

Signal Processing 23
Convolution 38
Convolutional Neural Networks 57
More Dimensions 72
Deeper Networks 81
Coding 87

Dirty secret of deep learning

As networks become larger, deep learning changes

We understand the rules that govern small networks (physics)

When the system becomes complex, we lose understanding (biology)

Early deep learning was inductive

Modern deep learning is deductive

To improve neural networks, we looked at:

- Deeper and wider networks
- New activation functions
- Parameter initialization
- New optimization methods

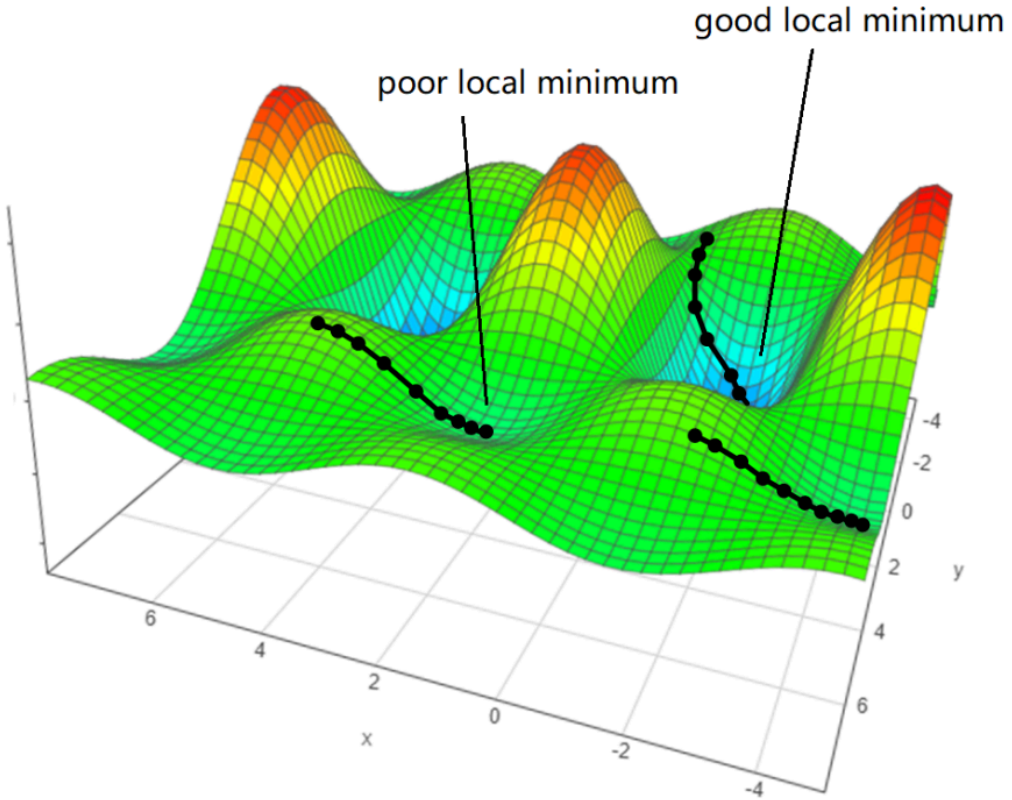
A 2-layer neural network can represent **any** continuous function to arbitrary precision

$$| f(\boldsymbol{x}, \boldsymbol{\theta}) - g(\boldsymbol{x}) | < \varepsilon$$

$$\lim_{d_h \to \infty} \varepsilon = 0$$

However, finding such $\boldsymbol{\theta}$ is a much harder problem

Gradient descent only guarantees convergence to a **local** optima

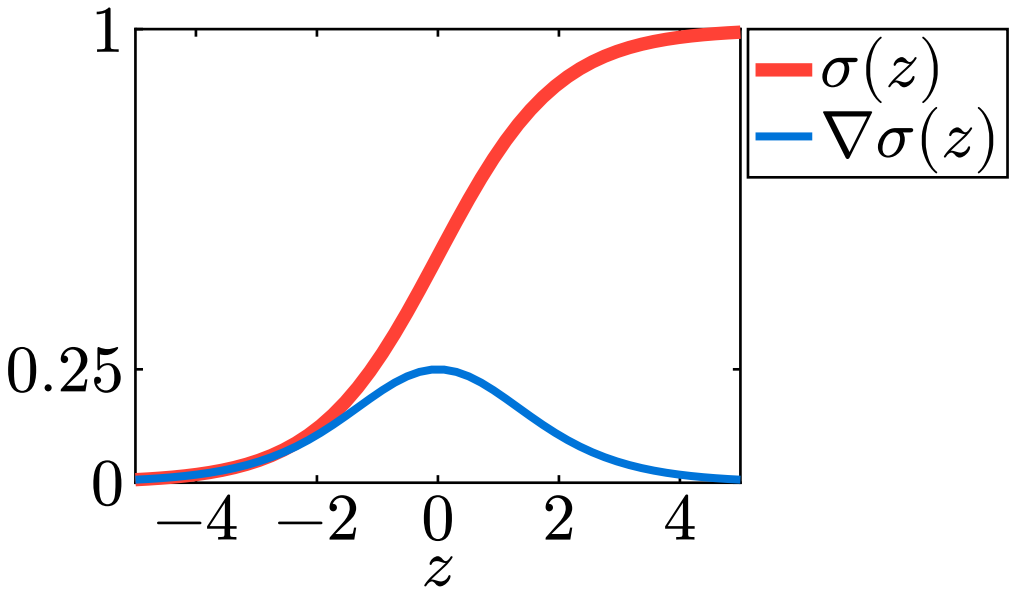


Deeper and wider networks often perform better

We call these **overparameterized** networks

"The probability of finding a "bad" (high value) local minimum is non-zero for small-size networks and decreases quickly with network size" - Choromanska, Anna, et al. *The loss surfaces of multilayer networks.* (2014)

Next, we looked at activation functions



The sigmoid function can result in a **vanishing gradient**

$$f(\boldsymbol{x},\boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}_3^\top \sigma(\boldsymbol{\theta}_2^\top \sigma(\boldsymbol{\theta}_1^\top \boldsymbol{x})))$$

$$\left(\nabla_{\boldsymbol{\theta}_1} f\right)(\boldsymbol{x},\boldsymbol{\theta}) = \underbrace{(\nabla\sigma)(\boldsymbol{\theta}_3^\top \sigma(\boldsymbol{\theta}_2^\top \sigma(\boldsymbol{\theta}_1^\top \boldsymbol{x})))}_{<0.25}\boldsymbol{\theta}_3 \underbrace{(\nabla\sigma)(\boldsymbol{\theta}_2^\top \sigma(\boldsymbol{\theta}_1^\top \boldsymbol{x}))}_{<0.25}\boldsymbol{\theta}_2 \underbrace{(\nabla\sigma)(\boldsymbol{\theta}_1^\top \boldsymbol{x})}_{<0.25}\boldsymbol{x}$$

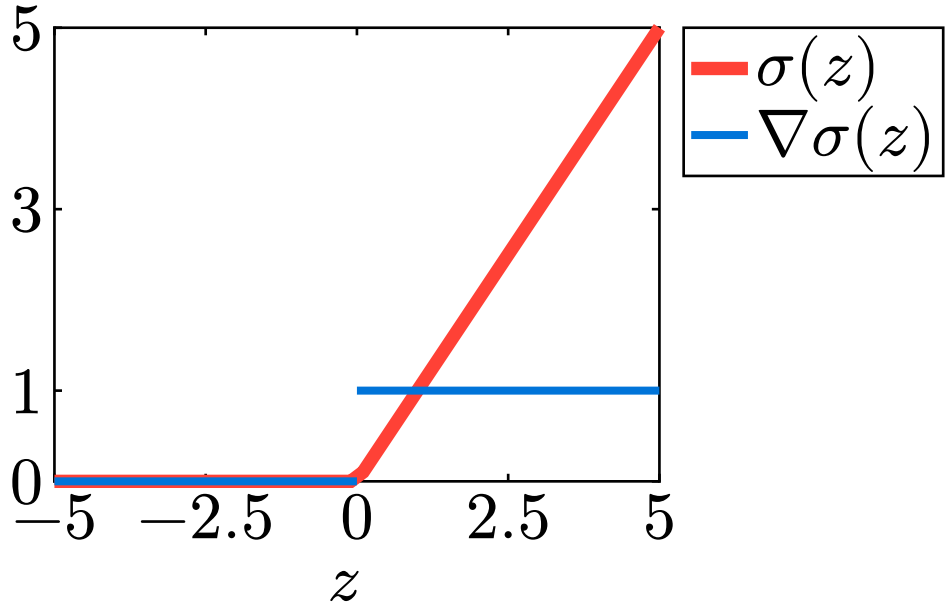
$$\max_{\boldsymbol{\theta},\boldsymbol{x}} \left(\nabla_{\boldsymbol{\theta}_1} f\right)(\boldsymbol{x},\boldsymbol{\theta}) \approx 0.01 \cdot \ldots$$

$$\lim_{\ell\to\infty} \left(\nabla_{\boldsymbol{\theta}_1} f\right)(\boldsymbol{x},\boldsymbol{\theta}) = 0$$

To fix the vanishing gradient, use the **rectified linear unit (ReLU)**

$$\sigma(z) = \max(0, z)$$

$$\nabla\sigma(z) = \begin{cases} 0 \text{ if } z < 0 \\ 1 \text{ if } z \geq 0 \end{cases}$$

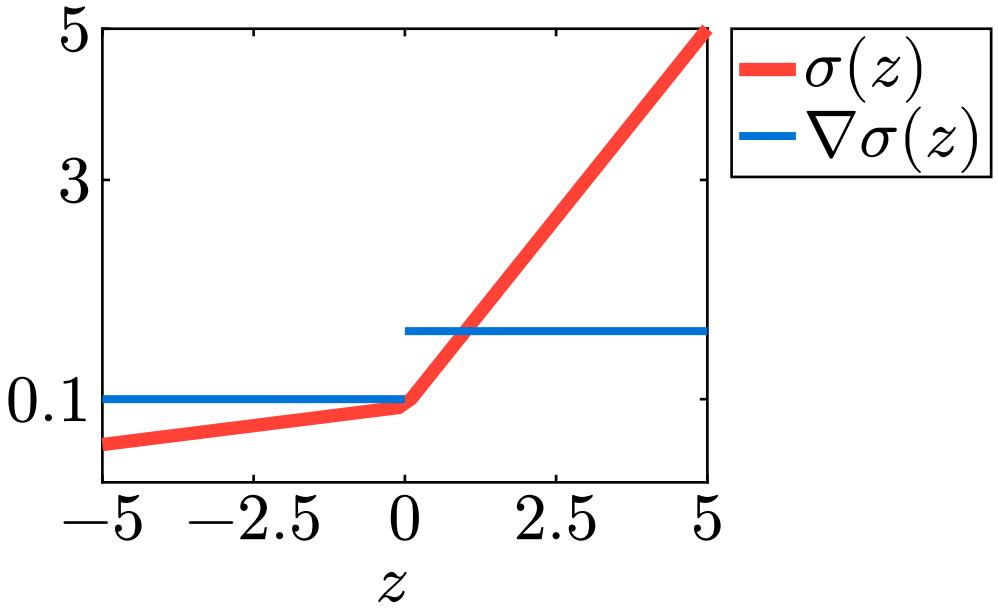


"Dead" neurons always output 0, and cannot recover

To fix dying neurons, use **leaky ReLU**

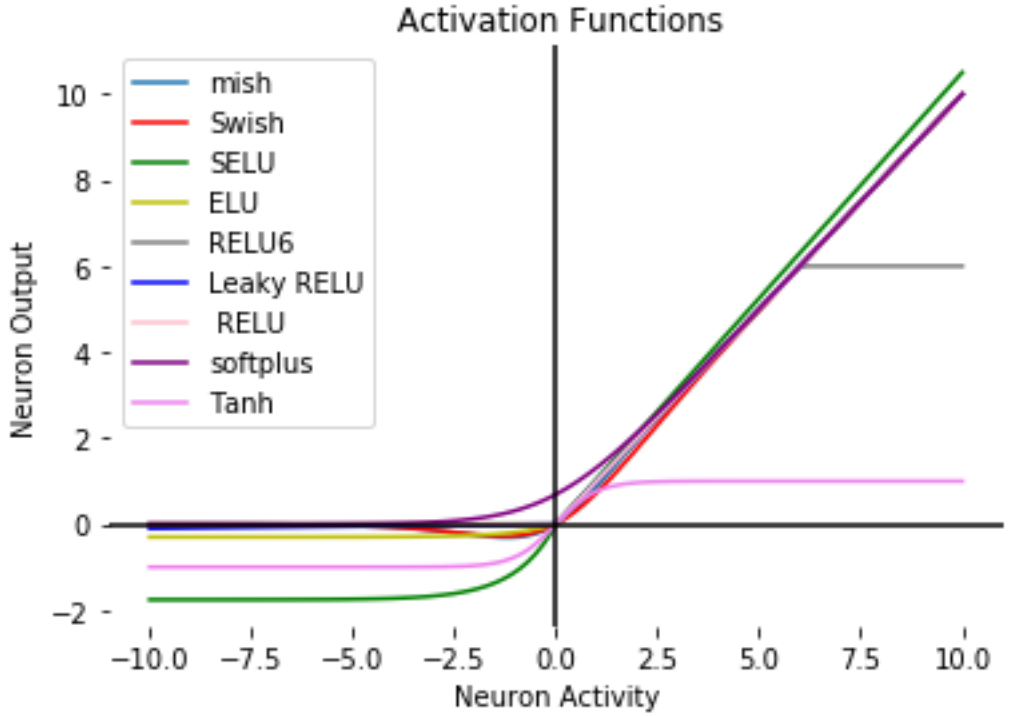
$$\sigma(z) = \max(0.1z, z)$$

$$\nabla\sigma(z) = \begin{cases} 0.1 \text{ if } z < 0 \\ 1 \text{ if } z \geq 0 \end{cases}$$



Small negative slope prevents dead neurons (nonzero slope everywhere)

As long as one neuron is positive in each layer, non-vanishing gradient


Activation Functions
mish
Swish
SELU
ELU
RELU6
Leaky RELU
RELU
softplus
Tanh
Neuron Output
10
8
6
4
2
0
−2
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
Neuron Activity

Then, we discussed parameter initialization

Initialization should:

1. Be random to break symmetry within a layer
2. Scale the parameters to prevent vanishing or exploding gradients

$$\boldsymbol{\theta} \sim \mathcal{U}\left[-\frac{\sqrt{6}}{\sqrt{2d_h}}, \frac{\sqrt{6}}{\sqrt{2d_h}}\right]$$

$$\boldsymbol{\theta} \sim \mathcal{U}\left[-\frac{\sqrt{6}}{\sqrt{d_x + d_y}}, \frac{\sqrt{6}}{\sqrt{d_x + d_y}}\right]$$

Finally, we reviewed improvements to optimization:

1. Stochastic gradient descent
2. Adaptive optimization
3. Weight decay

Stochastic gradient descent (SGD) reduces the memory usage

We cannot fit the full gradient in memory

In expectation, the stochastic gradient approximates the true gradient

$$\mathbb{E}_{\boldsymbol{X}_i \sim \boldsymbol{X}, \boldsymbol{Y}_i \sim \boldsymbol{Y}}[(\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}_i, \boldsymbol{Y}_i, \boldsymbol{\theta})] = (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$$

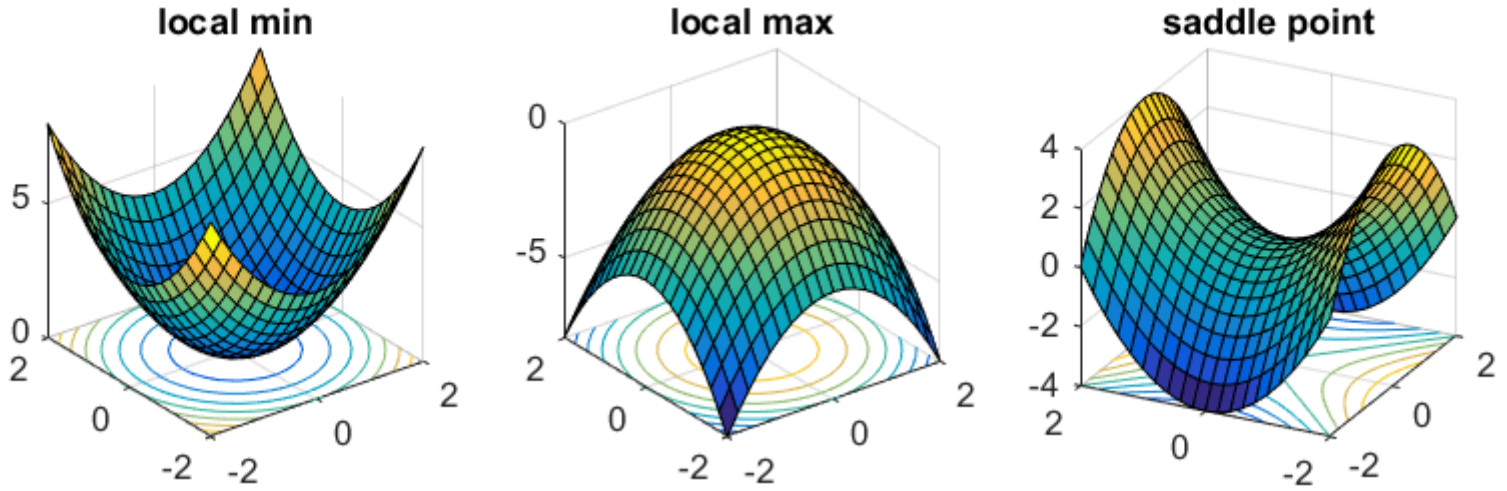
Note: Only equal under infinitely many samples $\boldsymbol{X}_i, \boldsymbol{Y}_i$

- Approximate expectation using finite samples

$$\hat{\mathbb{E}}_{\boldsymbol{X}_i \sim \boldsymbol{X}, \boldsymbol{Y}_i \sim \boldsymbol{Y}}[(\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}_i, \boldsymbol{Y}_i, \boldsymbol{\theta})] \approx (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$$

Stochastic gradient approximation adds noise during optimization

This noise can prevent premature convergence to bad optima



Use `torch.utils.data.DataLoader` for SGD

```
import torch
dataloader = torch.utils.data.DataLoader(
    training_data,
    batch_size=32, # How many datapoints to sample
    shuffle=True, # Randomly shuffle each epoch
)
for epoch in number_of_epochs:
    for batch in dataloader:
        X_j, Y_j = batch
        loss = L(X_j, Y_j, theta)
        ...
```

Then, we looked at adaptive variants of gradient descent

procedure GRADIENT DESCENT($\boldsymbol{X}$, $\boldsymbol{Y}$, $\mathcal{L}$, t, α)
  $\boldsymbol{\theta} \leftarrow$ Init()
  for $i \in \{1...t\}$ **do**
    ▷ Compute the gradient of the loss
    $\boldsymbol{J} \leftarrow (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$
    ▷ Update the parameters using the negative gradient
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \boldsymbol{J}$
  end
  return $\boldsymbol{\theta}$
end

Introduce **momentum**

function Momentum Gradient Descent($\boldsymbol{X}$, $\boldsymbol{Y}$, $\mathcal{L}$, t, α, β)
  $\boldsymbol{\theta} \leftarrow \text{Glorot}()$
  $\boldsymbol{M} \leftarrow \boldsymbol{0}$ # Init momentum
  for $i \in 1...t$ **do**
    $\boldsymbol{J} \leftarrow (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$ # Represents acceleration
    $\boldsymbol{M} \leftarrow \beta \cdot \boldsymbol{M} + (1 - \beta) \cdot \boldsymbol{J}$ # Momentum and acceleration
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \boldsymbol{M}$ # Multiply by momentum not gradient
  end
  return $\boldsymbol{\theta}$
end

Introduce **adaptive learning rate**

procedure RMSProp($\boldsymbol{X}$, $\boldsymbol{Y}$, $\mathcal{L}$, t, α, β)
  $\boldsymbol{\theta} \leftarrow \text{Glorot}()$
  $\boldsymbol{V} \leftarrow \boldsymbol{0}$ # Init variance
  for $i \in 1...t$ **do**
    $\boldsymbol{J} \leftarrow (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$ # Represents acceleration
    $\boldsymbol{V} \leftarrow \beta \cdot \boldsymbol{V} + (1 - \beta) \cdot \boldsymbol{J} \odot \boldsymbol{J}$ # Magnitude
    $\triangleright$ # Rescale grad by magnitude of prev updates
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \boldsymbol{J} \oslash \boldsymbol{V}^{\odot \frac{1}{2}}$
  end
  return $\boldsymbol{\theta}$
end

Combine **momentum** and **adaptive learning rate** to create **Adam**

procedure ADAPTIVE MOMENT ESTIMATION($\boldsymbol{X}$, $\boldsymbol{Y}$, $\mathcal{L}$, t, α, β_1, β_2)
  $\boldsymbol{\theta} \leftarrow \text{Glorot}()$
  $\boldsymbol{M}, \boldsymbol{V} \leftarrow \boldsymbol{0}$
  for $i \in 1...t$ **do**
    $\boldsymbol{J} \leftarrow (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$
    $\boldsymbol{M} \leftarrow \beta_1 \cdot \boldsymbol{M} + (1 - \beta_1) \cdot \boldsymbol{J}$ # Update momentum
    $\boldsymbol{V} \leftarrow \beta_2 \cdot \boldsymbol{V} + (1 - \beta_2) \cdot \boldsymbol{J} \odot \boldsymbol{J}$ # Update magnitude
    ▷ # Rescale momentum by magnitude
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \boldsymbol{M} \oslash \boldsymbol{V}^{\odot \frac{1}{2}}$
  end
  return $\boldsymbol{\theta}$ # Note, we use biased $\boldsymbol{M}, \boldsymbol{V}$ for clarity
end

Many modern optimizers also include **weight decay**

Weight decay penalizes large parameters

$$\mathcal{L}_{\text{decay}}(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta}) = \mathcal{L}_{\text{decay}}(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta}) + \lambda \sum_i \theta_i^2$$

Creates smoother, parabolic loss landscape that is easier to optimize

Let's finish coding!

https://colab.research.google.com/drive/1D8laihsmlfwozsoEooHo5-Ocn_F1iXtj?usp=sharing

Signal Processing

Signal Processing

Our neural networks do not consider the structure of data

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{d_x} \end{bmatrix}$$

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{d_x} \end{bmatrix} \qquad \boldsymbol{\theta} \stackrel{?}{=} \begin{bmatrix} \theta_7 \\ \theta_4 \\ \vdots \\ \theta_1 \end{bmatrix}$$

Signal Processing

We assume no relationship or ordering for input elements or parameter

$$\sigma(\theta_1 x_1 + \theta_2 x_2) \Rightarrow \sigma(\theta_2 x_2 + \theta_1 x_1)$$



These images are equivalent to a perceptron!

Signal Processing

The real world is structured

Our models should use this structure

- Better data efficiency
- Better generalization

To do so, we will think of the world as a collection of signals

Signal Processing

A **signal** represents information as a function of time, space or some other variable

$$x(t) = 2t + 1$$

$$x(u, v) = \frac{u^2}{v} - 3$$

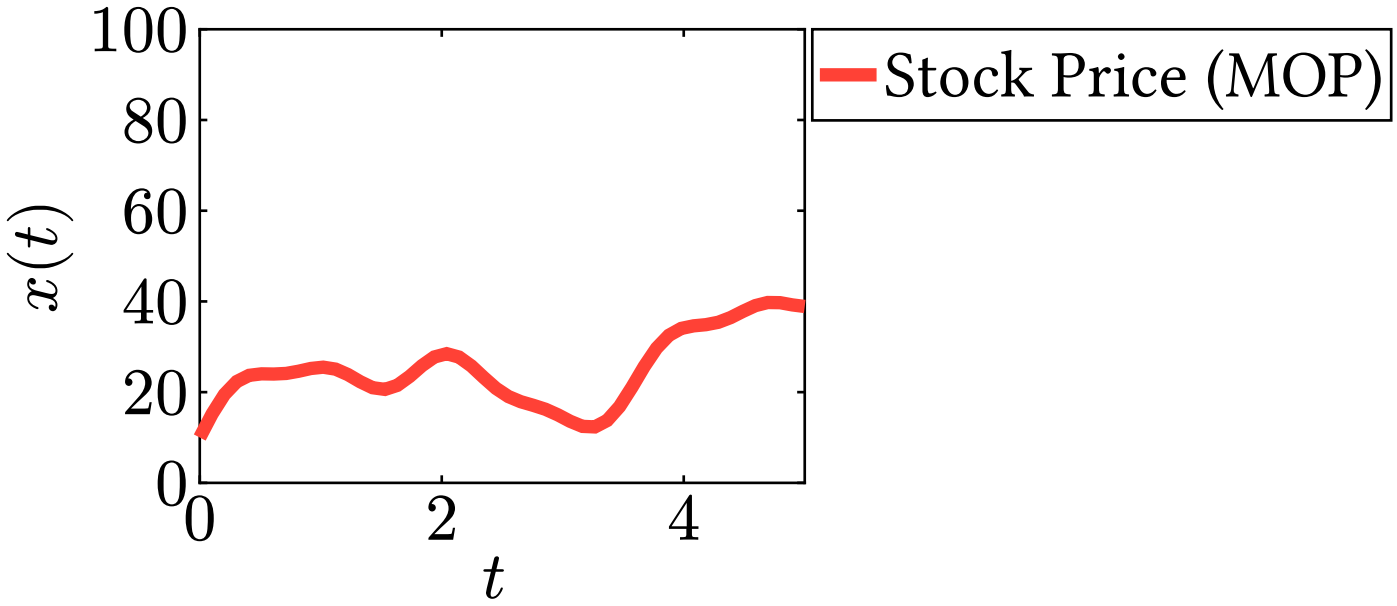
$x(t), x(u, v)$ represent physical processes that we may or may not know

In **signal processing**, we analyze the meaning of signals

Knowing the meaning of signals is very useful

Signal Processing

$$x(t) = \text{stock price}$$

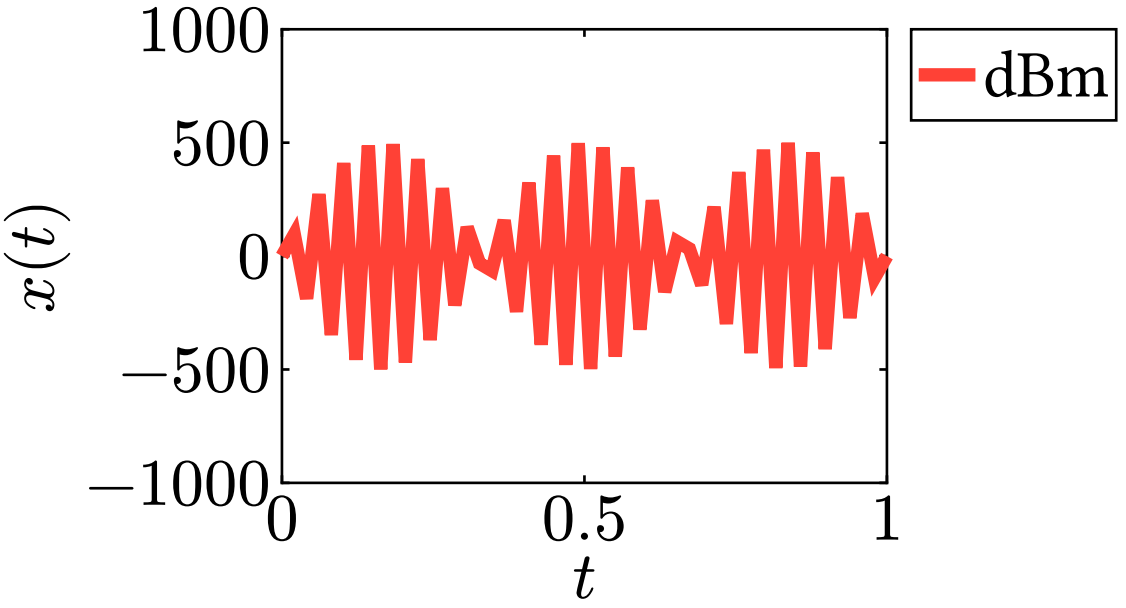


There is an underlying structure to $x(t)$

Structure: Tomorrow's stock price will be close to today's stock price

Signal Processing

$$x(t) = \text{audio}$$

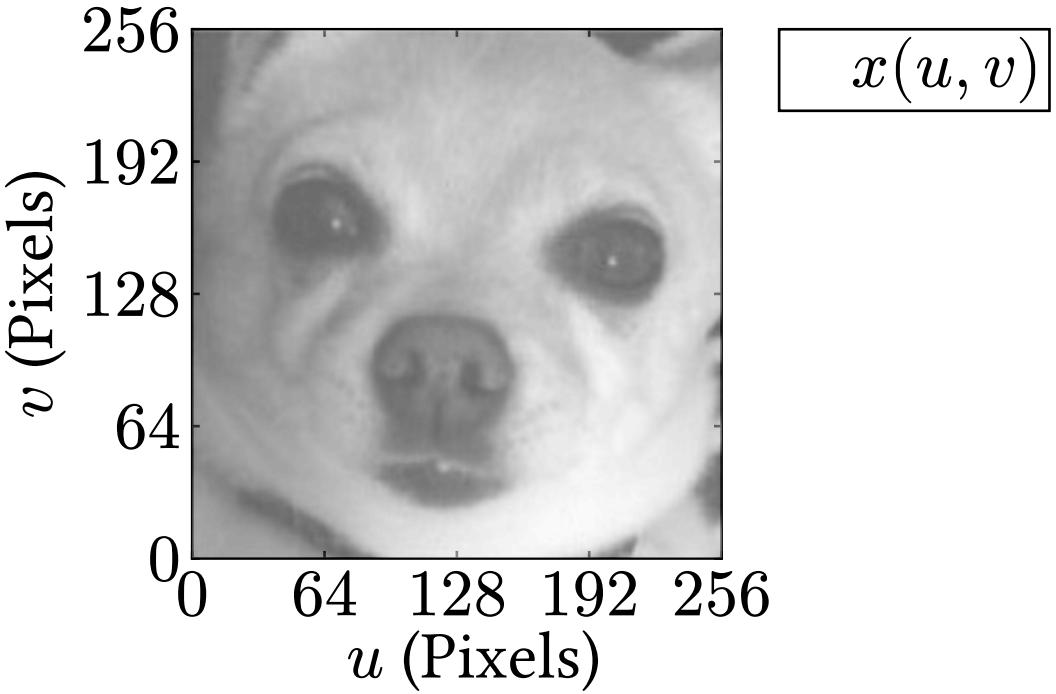


Structure: Nearby waves form syllables

Structure: Nearby syllables combine to create meaning

Signal Processing

$$x(u, v) = \text{image}$$



Structure: Repeated components (circles, symmetry, eyes, nostrils, etc)

Signal Processing

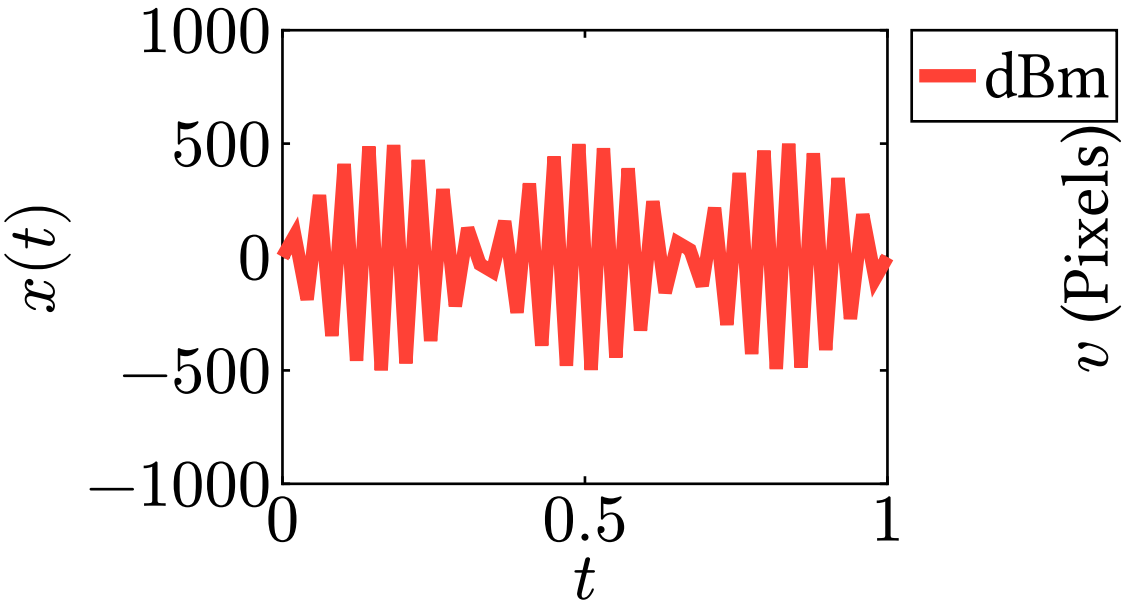
In signal processing, we often consider two properties:

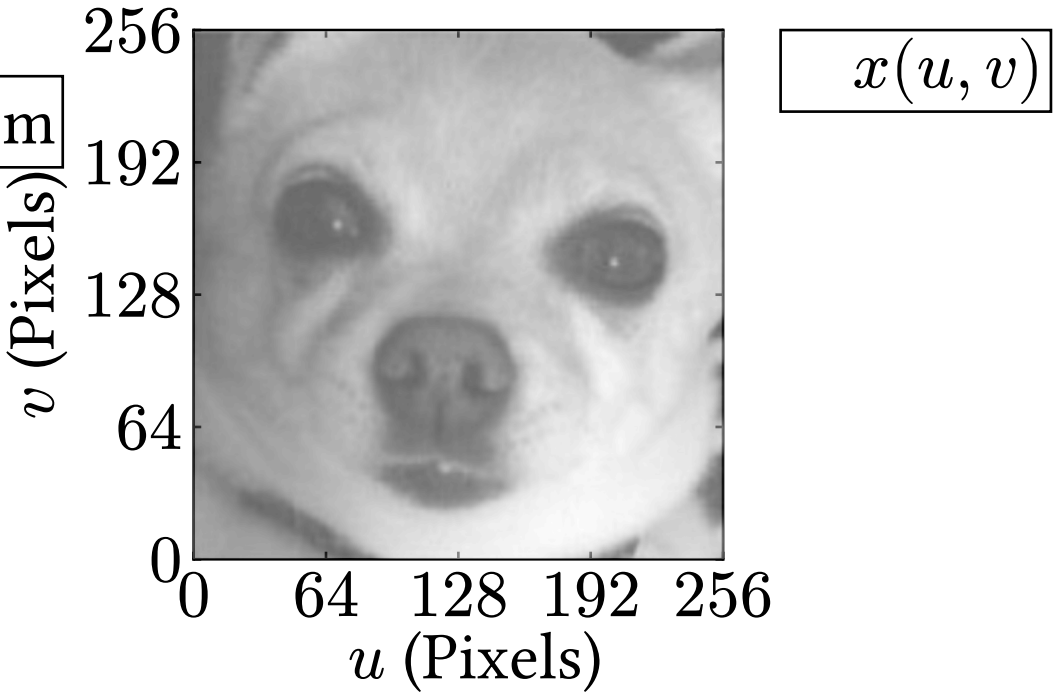
- Locality
- Translation equivariance

Signal Processing

Locality: Information structured and concentrated over small regions

$$\underbrace{x(t+\varepsilon) \approx f(x(t)) + \varepsilon}_{\text{Nearby regions related}} \qquad \underbrace{y = f(x(s)), \quad s \in (-\varepsilon, \varepsilon)}_{\text{Information concentrated in small region}}$$

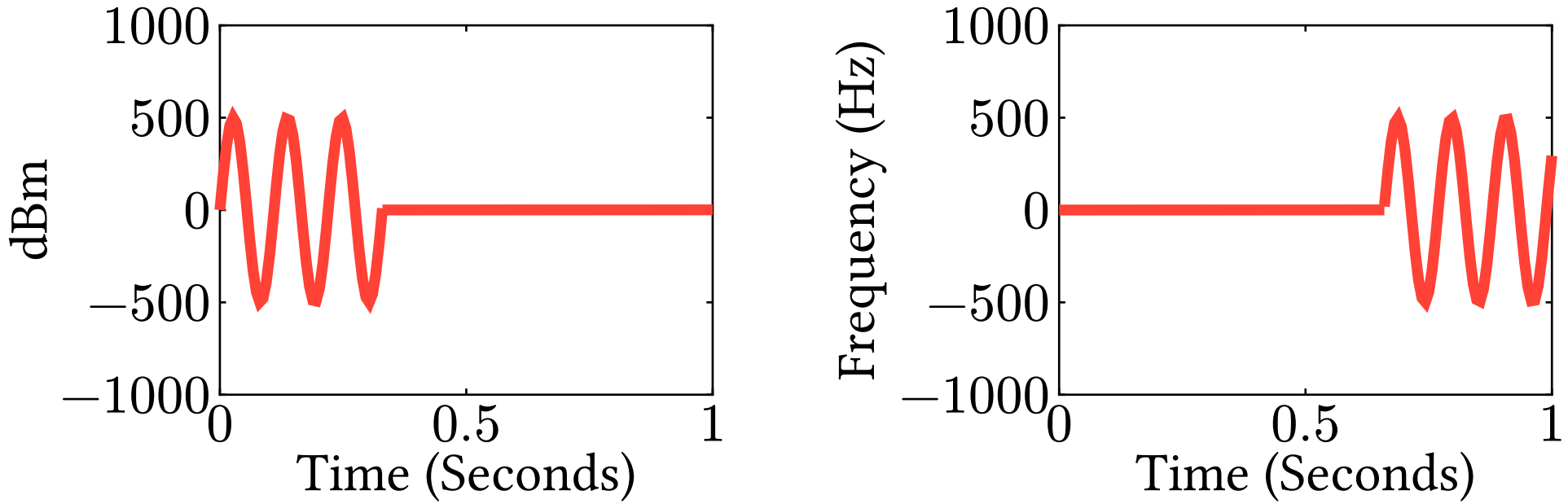




Signal Processing

Translation Equivariance: Shift in signal results in shift in output

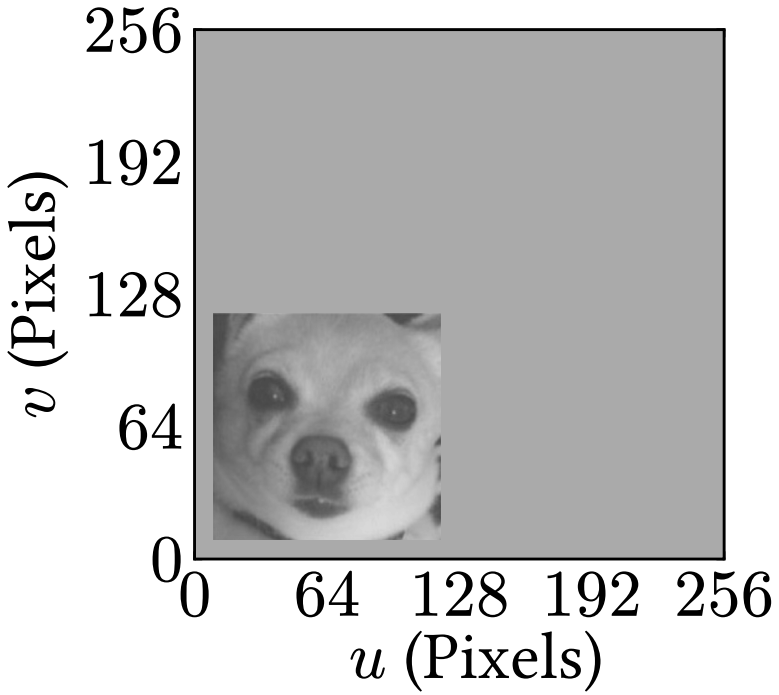
$$f(x(t+\tau)) = y(t+\tau)$$

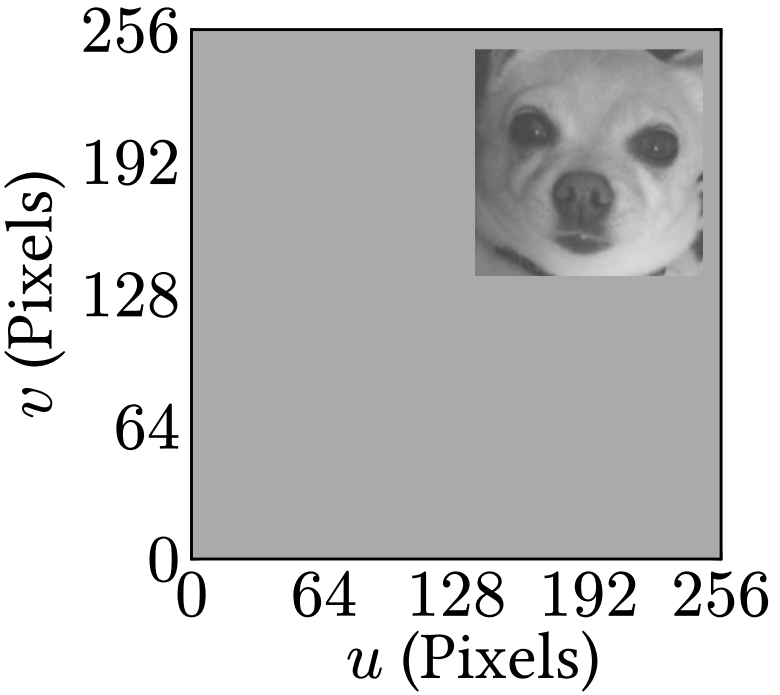


Both say “hello”

Signal Processing

Translation Equivariance: Shift in signal results in shift in output

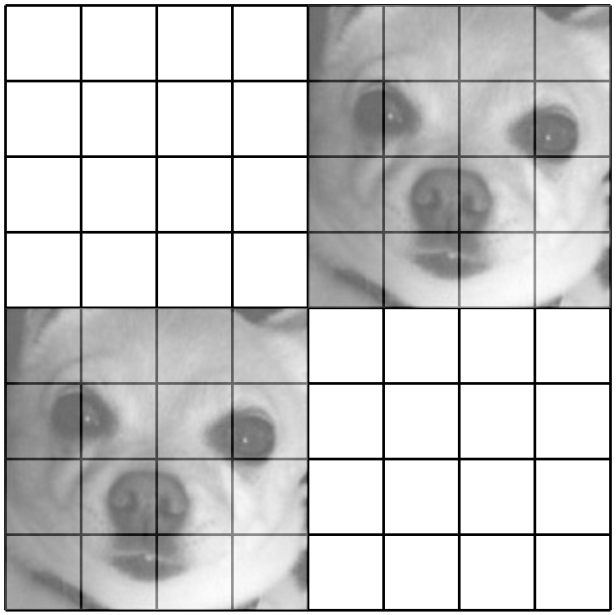




Both contain a dog

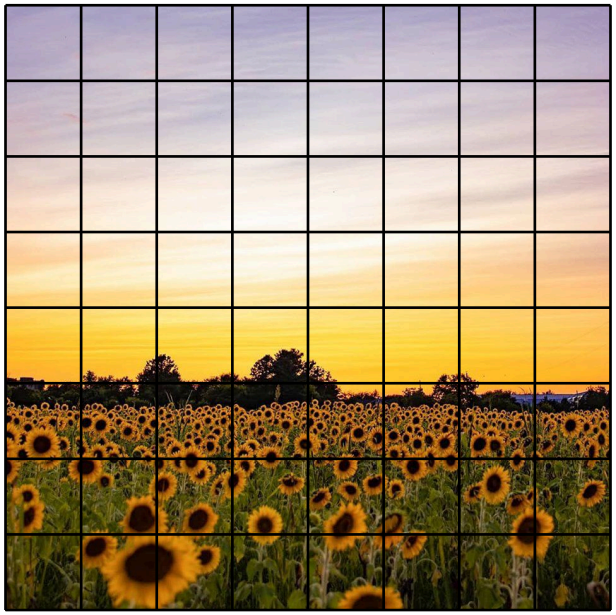
Signal Processing

Perceptrons are not local or translation equivariant, each pixel is an independent neuron



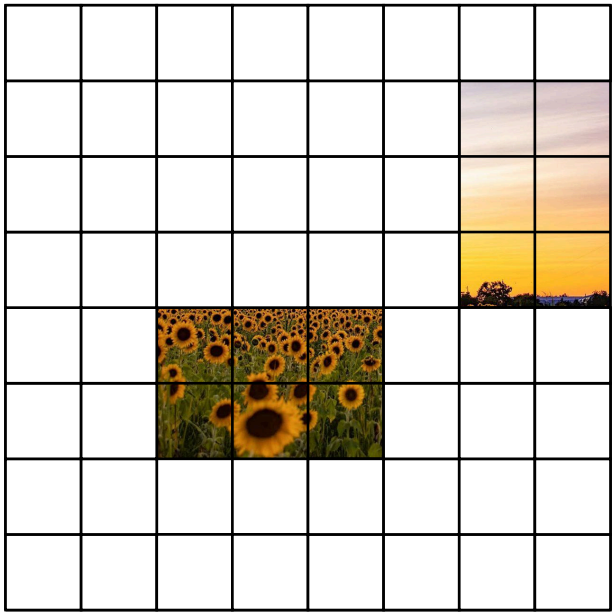
Signal Processing

A more realistic scenario of locality and translation equivariance



Signal Processing

A more realistic scenario of locality and translation equivariance



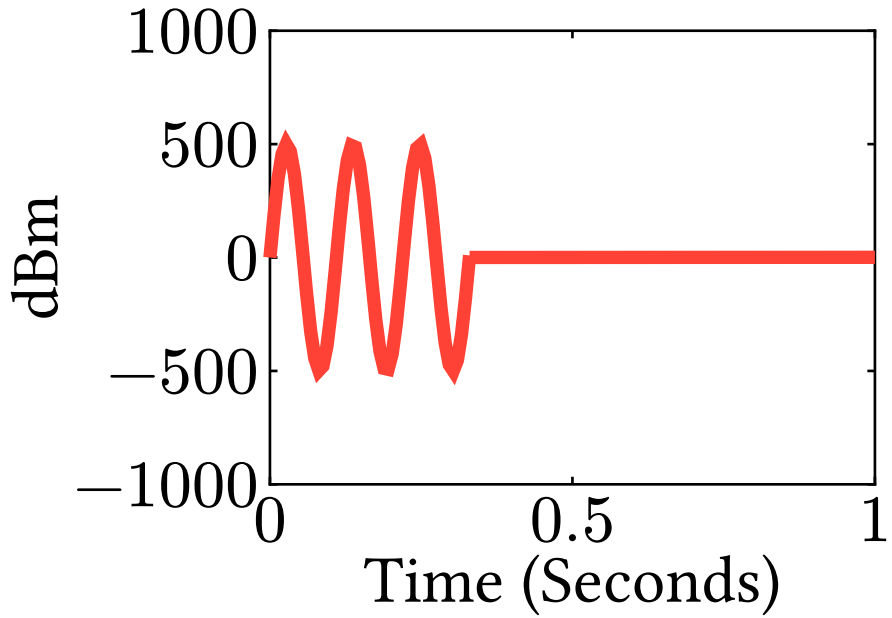
Our brains use the world's structure to infer missing information

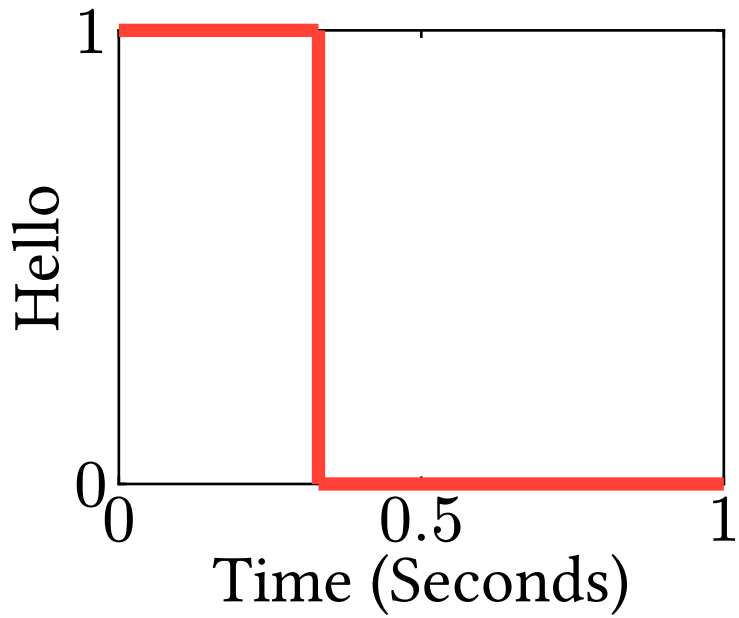
Our neural networks should do the same

Convolution

Convolution

In signal processing, we often turn signals into other signals





A standard way to transform signals is **convolution**

Convolution is translation equivariant and local

Convolution

Convolution is the sum of products of a signal $x(t)$ and a **filter** $g(t)$

If the t is continuous in $x(t)$

$$x(t) * g(t) = \int_{-\infty}^{\infty} x(t-\tau)g(\tau)d\tau$$

If the t is discrete in $x(t)$

$$x(t) * g(t) = \sum_{\tau=-\infty}^{\infty} x(t-\tau)g(\tau)$$

We slide the filter $g(t)$ across the signal $x(t)$

Convolution

Note: Convolution slides the filter from **right to left**

$$x(t) * g(t) = \int_{-\infty}^{\infty} x(t-\tau)g(\tau)d\tau = ... + x(t+10)g(t) + x(t+9)g(t) + ...$$

$$x(t) * g(t) = \sum_{\tau=-\infty}^{\infty} x(t-\tau)g(\tau) \quad = ... + x(t+10)g(t) + x(t+9)g(t) + ...$$

This is equivalent to flipping the filter and sliding left to right

$$g_{\text{flip}}(t) = g(-t)$$

Convolution

We will assume all filters $g(t)$ are "pre-flipped" in this course

- Simply scan the filter $g(t)$ left to right
- Easier to understand and how we implement it in neural networks

$$x(t) * g(t) = \int_{-\infty}^{\infty} x(t+\tau)g(\tau)d\tau = ... + x(t+1)g(1) + x(t+2)g(2) + ...$$

$$x(t) * g(t) = \sum_{\tau=-\infty}^{\infty} x(t+\tau)g(\tau) \quad = ... + x(t+1)g(1) + x(t+2)g(2) + ...$$

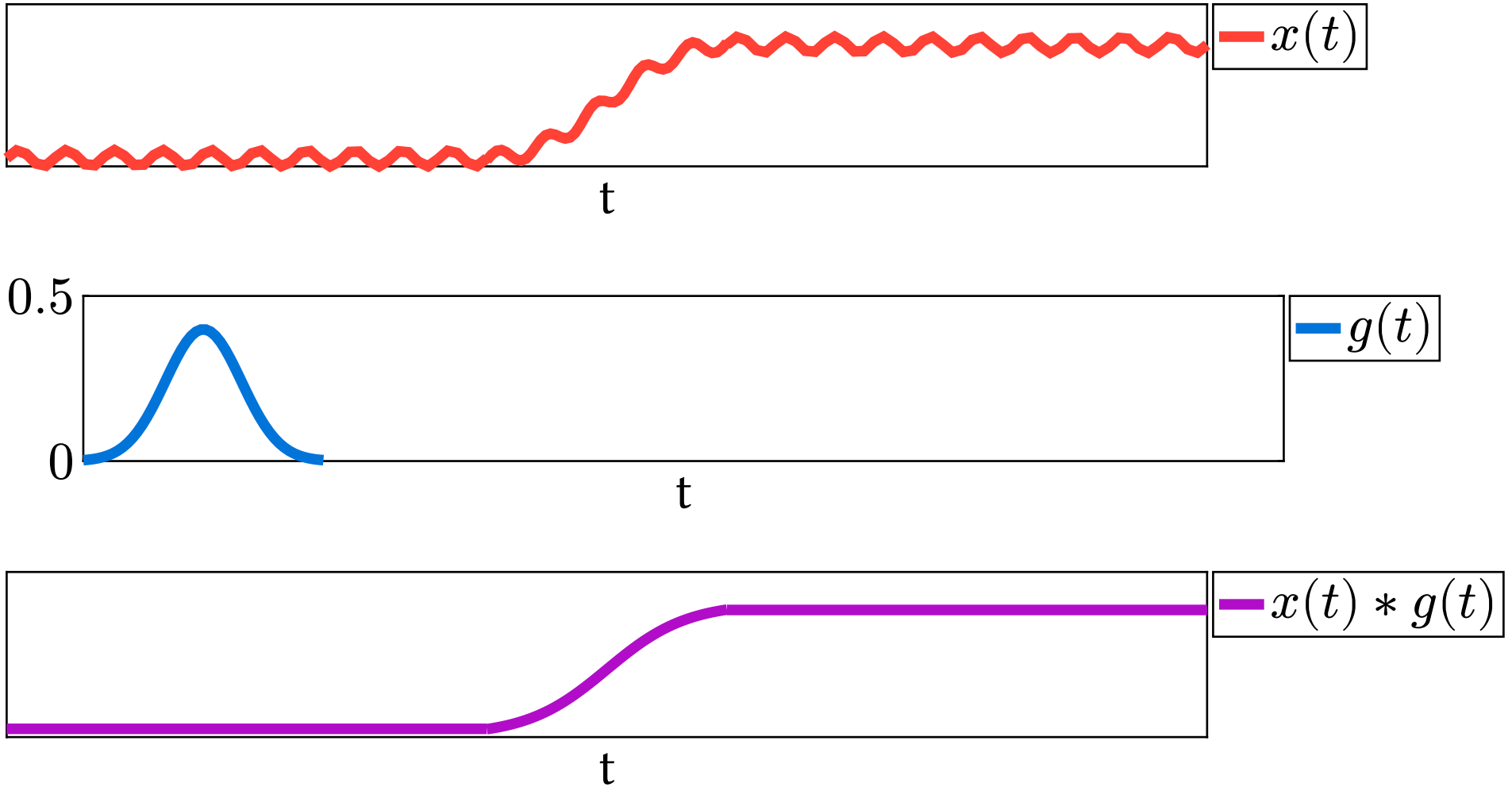
Confused? Let us do an example

Convolution

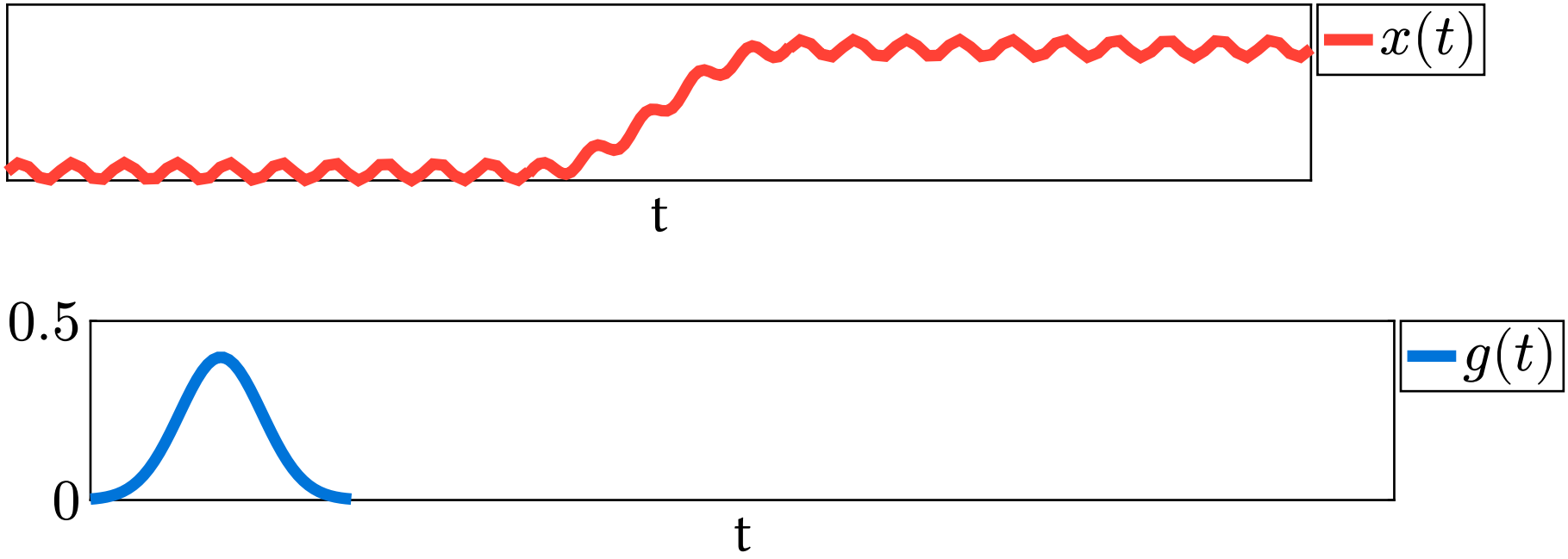
Example: Let us examine a low-pass filter

The filter will take a signal and remove noise, producing a cleaner signal

Convolution



Convolution



Convolution is **local** to the filter $g(t)$

Convolution is also **equivariant** to time/space shifts

Convolution

We use continuous convolution for analog signals

- Physics
- Control theory
- Electrical engineering

Deep learning uses discrete convolution (digital hardware)

- Images
- Recorded audio
- Anything stored as bits

It is good to know both! Continuous for theory, discrete for software

Let us do a discrete example, I think it will make convolution very clear

Convolution

Example: Discrete convolution with filter size 2

$$y(t) = x(t) * g(t) = \sum_{\tau=0}^{1} x(t+\tau)g(\tau)$$

$$y(t) = x(t)g(0) + x(t+1)g(1)$$

$$\begin{bmatrix} x(t) \\ g(t) \\ y(t) \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 1 & & & \\ & & & & \end{bmatrix}$$

Convolution

$$y(t) = x(t) * g(t) = \sum_{\tau=0}^{1} x(t+\tau)g(\tau)$$

$$y(t) = x(t)g(0) + x(t+1)g(1)$$

$$\begin{bmatrix} x(t) \\ g(t) \\ y(t) \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 1 & & & \\ 4 & & & & \end{bmatrix}$$

Convolution

$$y(t) = x(t) * g(t) = \sum_{\tau=0}^{1} x(t+\tau)g(\tau)$$

$$y(t) = x(t)g(0) + x(t+1)g(1)$$

$$\begin{bmatrix} x(t) \\ g(t) \\ y(t) \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ & 2 & 1 & & \\ 4 & 7 & & & \end{bmatrix}$$

Convolution

$$y(t) = x(t) * g(t) = \sum_{\tau=0}^{1} x(t+\tau)g(\tau)$$

$$y(t) = x(t)g(0) + x(t+1)g(1)$$

$$\begin{bmatrix} x(t) \\ g(t) \\ y(t) \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ & & 2 & 1 & \\ 4 & 7 & 10 & & \end{bmatrix}$$

Convolution

$$y(t) = x(t) * g(t) = \sum_{\tau=0}^{1} x(t+\tau)g(\tau)$$

$$y(t) = x(t)g(0) + x(t+1)g(1)$$

$$\begin{bmatrix} x(t) \\ g(t) \\ y(t) \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ & & & 2 & 1 \\ 4 & 7 & 10 & 13 & \end{bmatrix}$$

Convolution

$$\begin{bmatrix} x(t) \\ g(t) \\ y(t) \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 1 & & & \\ 4 & 5 & 10 & 13 & \end{bmatrix}$$

Question: Does anybody see a connection to neural networks?

Hint: What if I rewrite the filter?

$$\begin{bmatrix} x(t) \\ g(t) \\ y(t) \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ \theta_1 & \theta_2 & & & \\ & & & & \end{bmatrix}$$

Convolution

$$y(t) = x(t) * g(t) = \sum_{\tau=0}^{1} x(t+\tau)g(\tau)$$

$$y(t) = x(t)g(0) + x(t+1)g(1)$$

$$\begin{bmatrix} x(t) \\ g(t) \\ y(t) \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ \theta_1 & \theta_2 & & & \\ \theta_1 + 2\theta_2 & & & & \end{bmatrix}$$

Convolution

$$y(t) = x(t) * g(t) = \sum_{\tau=0}^{1} x(t+\tau)g(\tau)$$

$$y(t) = x(t)g(0) + x(t+1)g(1)$$

$$\begin{bmatrix} x(t) \\ g(t) \\ y(t) \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ & \theta_1 & \theta_2 & & \\ \theta_1 + 2\theta_2 & 2\theta_1 + 3\theta_2 & & & \end{bmatrix}$$

Convolution

$$\begin{bmatrix} x(t) \\ g(t) \\ y(t) \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ & \theta_1 & \theta_2 & & \\ \theta_1 + 2\theta_2 & 2\theta_1 + 3\theta_2 & & & \end{bmatrix}$$

Just like neural networks, convolution is a linear operation

It is a weighted sum of the inputs, just like a neuron

$$x(t) * g(t) = \sum_{\tau=0}^{d_x} x(t+\tau) \cdot g(\tau)$$

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \sum_{i=0}^{d_x} x_i \cdot \theta_i$$

Convolution

$$x(t) * g(t) = \sum_{\tau=0}^{d_x} x(t+\tau) \cdot g(\tau)$$

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \sum_{i=0}^{d_x} x_i \cdot \theta_i$$

Question: How does convolution differ from a neuron?

Answer: In a neuron, each input x_i has a different parameter θ_i. In convolution, we reuse (slide) θ_i over $x_1, x_2, \ldots$

Convolutional Neural Networks

Convolutional Neural Networks

Convolutional neural networks (CNNs) use convolutional layers

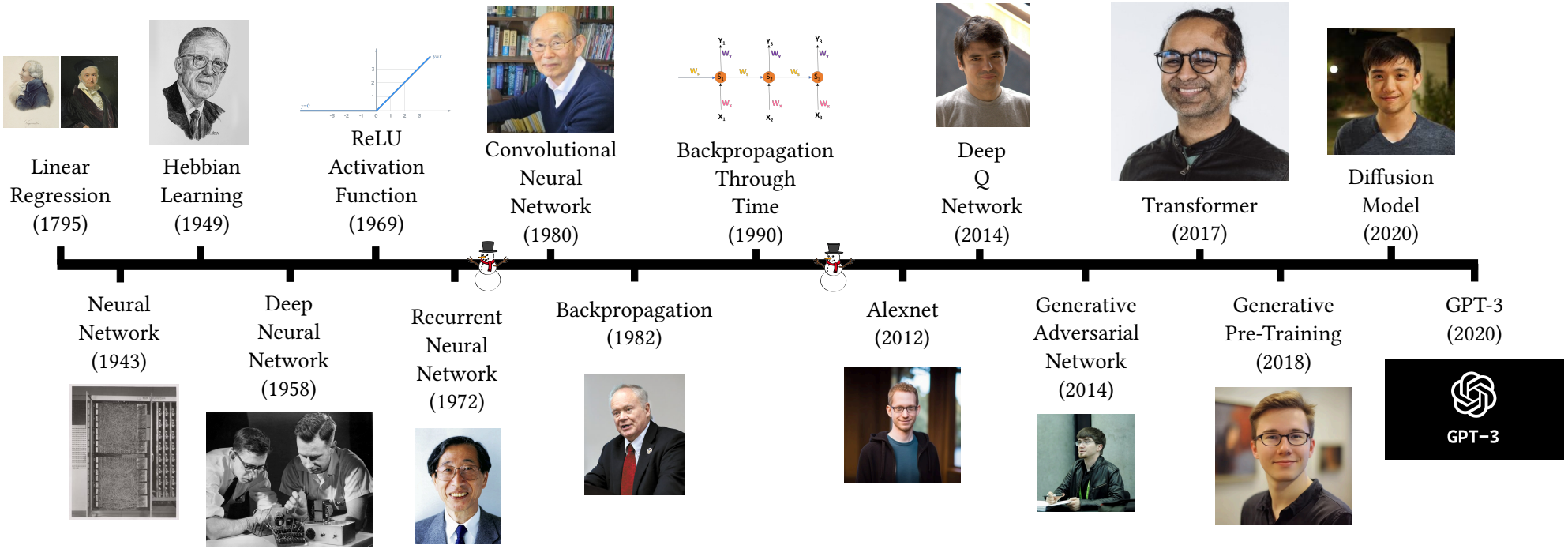
Their translation equivariance and locality make them very efficient

They also scale to variable-length sequences

Efficiently expands neural networks to images, videos, sounds, etc

Convolutional Neural Networks

CNNs have been around since the 1980's



2012: GPU and CNN efficiency resulted in breakthroughs

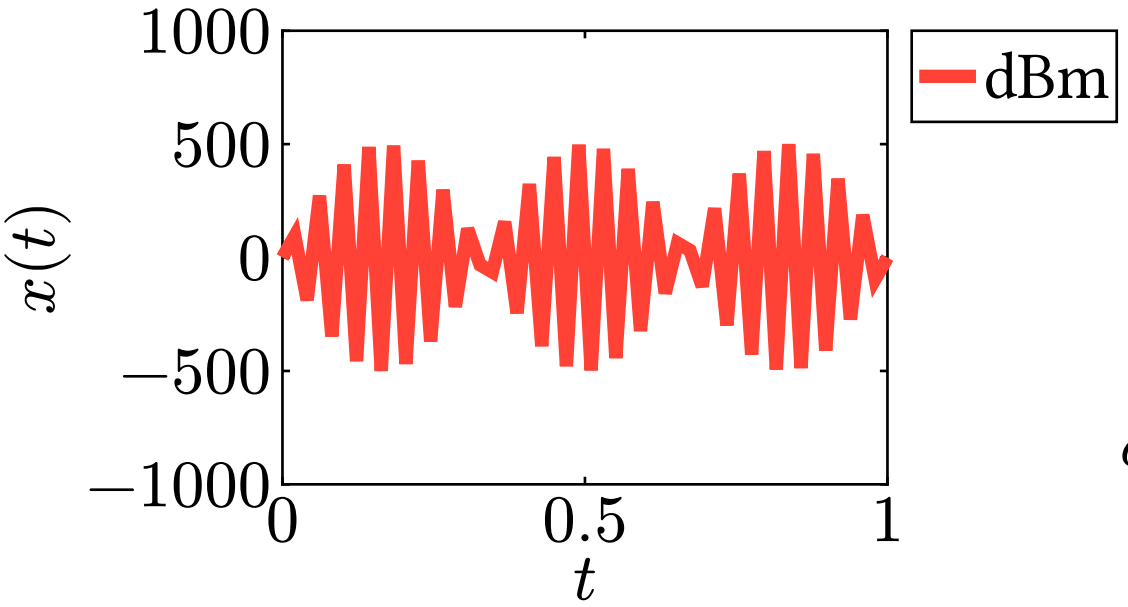
Convolutional Neural Networks

So how does a convolutional neural network work?

Convolutional Neural Networks

Recall the neuron

Neuron

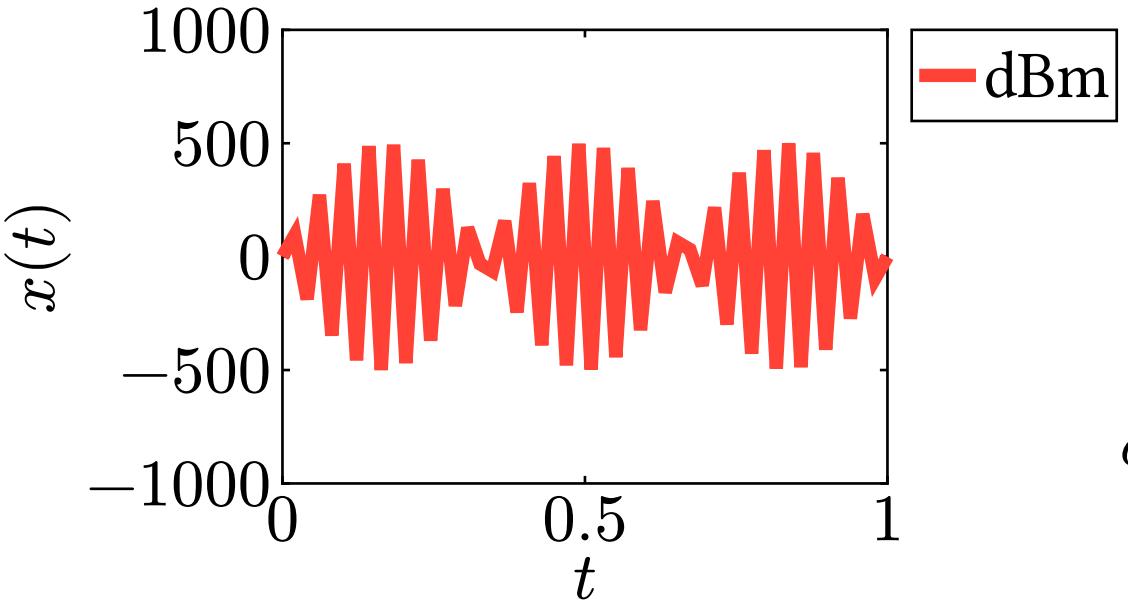


$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^\top \overline{\boldsymbol{x}})$$

$$f\left(\begin{bmatrix} x(0.1) \\ x(0.2) \\ \vdots \end{bmatrix}, \boldsymbol{\theta}\right) =$$

$$\sigma(\theta_0 + \theta_1 x(0.1) + \theta_2 x(0.2) + ...)$$

Convolutional Neural Networks



$$f\left(\begin{bmatrix} x(0.1) \\ x(0.2) \\ \vdots \end{bmatrix}, \boldsymbol{\theta}\right) =$$

$$\sigma(\theta_0 + \theta_1 x(0.1) + \theta_2 x(0.2) + ...)$$

Question: How many parameters do we need?

Answer 1: 10, parameters scale with sequence length

Question: What if sequence is 1.1 seconds long? **A:** Train new network

Convolutional Neural Networks

One parameter for each timestep does not work well

$$f\left(\begin{bmatrix} x(0.1) \\ x(0.2) \\ \vdots \end{bmatrix}, \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \end{bmatrix}\right)$$

$$= \sigma(\theta_0 + \theta_1 x(0.1) + \theta_2 x(0.2) + \theta_3 x(0.3) + \theta_4 x(0.4) + \theta_5 x(0.5) + ...)$$

Question: How many parameters if speech is 2 hours long? **A:** 72,000

Question: What if speech is 2 hours and 0.1 seconds long?

Answer: Train a new neural network

Convolutional Neural Networks

$$f\left(\begin{bmatrix} x(0.1) \\ x(0.2) \\ \vdots \end{bmatrix}, \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \end{bmatrix}\right)$$

$$= \sigma(\theta_0 + \theta_1 x(0.1) + \theta_2 x(0.2) + \theta_3 x(0.3) + \theta_4 x(0.4) + \theta_5 x(0.5) + ...)$$

What if we reuse parameters?

$$f\left(\begin{bmatrix} x(0.1) \\ x(0.2) \\ \vdots \end{bmatrix}, \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix}\right) = \begin{bmatrix} \sigma(\theta_0 + \theta_1 x(0.1) + \theta_2 x(0.2)) \\ \sigma(\theta_0 + \theta_1 x(0.2) + \theta_2 x(0.3)) \\ \sigma(\theta_0 + \theta_1 x(0.3) + \theta_2 x(0.4)) \\ \vdots \end{bmatrix}$$

Convolutional Neural Networks

$$f\left(\begin{bmatrix} x(0.1) \\ x(0.2) \\ \vdots \end{bmatrix}, \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix}\right) = \begin{bmatrix} \sigma(\theta_0 + \theta_1 x(0.1) + \theta_2 x(0.2)) \\ \sigma(\theta_0 + \theta_1 x(0.2) + \theta_2 x(0.3)) \\ \sigma(\theta_0 + \theta_1 x(0.3) + \theta_2 x(0.4)) \\ \vdots \end{bmatrix}$$

This is convolution, we slide our filter $g(t) = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix}$ over the input $x(t)$

Convolutional Neural Networks

We can write both a perceptron and convolution layer in similar forms

$$f(x(t), \boldsymbol{\theta}) = \underbrace{\sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.1) \\ x(0.2) \\ \vdots \end{bmatrix}\right)}_{\text{Perceptron}} \qquad f(x(t), \boldsymbol{\theta}) = \underbrace{\begin{bmatrix} \sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.1) \\ x(0.2) \end{bmatrix}\right) \\ \sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.2) \\ x(0.3) \end{bmatrix}\right) \\ \vdots \end{bmatrix}}_{\text{Convolution layer}}$$

A convolution layer applies a “mini” perceptron at each timestep

Convolutional Neural Networks

$$f(x(t), \boldsymbol{\theta}) = \begin{bmatrix} \sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.1) \\ x(0.2) \end{bmatrix}\right) \\ \sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.2) \\ x(0.3) \end{bmatrix}\right) \\ \vdots \end{bmatrix}$$

Local, only considers a few nearby inputs at a time

Translation equivariant, each output corresponds to input

Convolutional Neural Networks

$$f(x(t), \boldsymbol{\theta}) = \begin{bmatrix} \sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.1) \\ x(0.2) \end{bmatrix}\right) \\ \sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.2) \\ x(0.3) \end{bmatrix}\right) \\ \vdots \end{bmatrix}$$

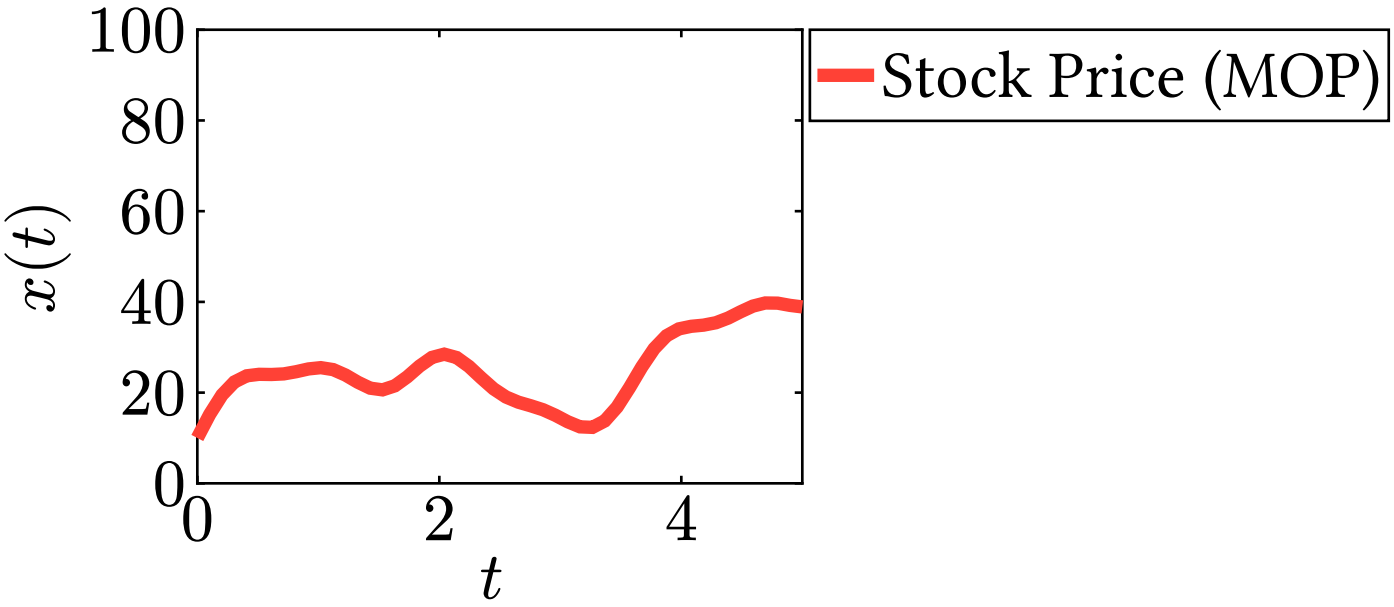
Question: What is the shape of the results?

Answer 1: Depends on sequence length and filter size!

Answer 2: $T - k + 1$, where T is sequence length and k is filter length

Convolutional Neural Networks

Maybe we want to predict a single output, e.g. stock price tomorrow



But convolutional layers output a variable-length signal $(T - k + 1)$!

Convolutional Neural Networks

If we want a single output, we should use **pooling**

$$z(t) = f(x(t), \boldsymbol{\theta}) = \left[\sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.1) \\ x(0.2) \end{bmatrix} \right) \quad \sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.2) \\ x(0.3) \end{bmatrix} \right) \quad \dots \right]^\top$$

$$\text{SumPool}(z(t)) = \sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.1) \\ x(0.2) \end{bmatrix} \right) + \sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.2) \\ x(0.3) \end{bmatrix} \right) + \dots$$

$$\text{MeanPool}(z(t)) = \frac{\text{SumPool}(z(t))}{T - k + 1}; \quad \text{MaxPool}(z(t)) = \max(z(t))$$

Convolutional Neural Networks

Question: $x(t)$ is a function, what is the function signature?

Answer:

$$x : \mathbb{R}_+ \mapsto \mathbb{R}$$

So far, we have considered:

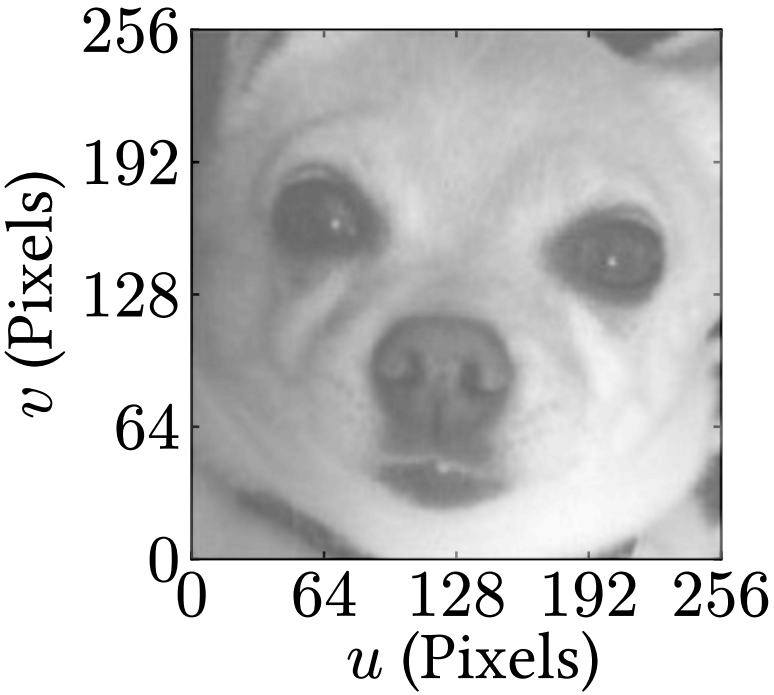
- 1 dimensional variable t
- 1 dimensional output/channel $x(t)$
- 1 filter

We must consider a more general case

Things will get more complicated, but the core idea is exactly the same

More Dimensions

More Dimensions



$$x(u, v)$$

Question: How many input dimensions for x?

Answer: 2: u, v

Question: How many output dimensions/channels for x?

Answer: 1, black/white value

$$x : \underbrace{\{0...255\}}_{\text{width}} \times \underbrace{\{0...255\}}_{\text{height}} \mapsto \underbrace{[0, 1]}_{\text{Color values}}$$

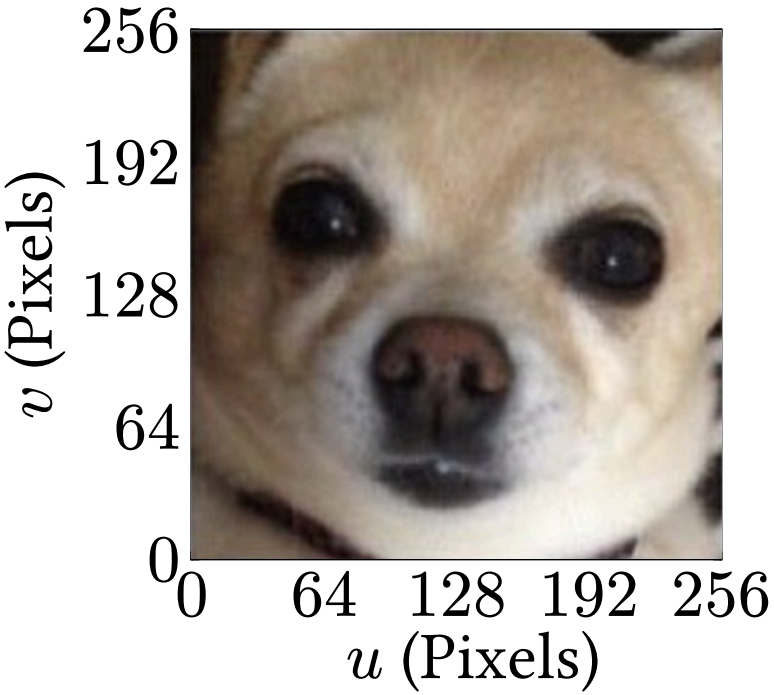
More Dimensions

0	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1
1	0	0	1	1	0	0	1
1	0	0	1	1	0	0	1
1	1	0	0	0	1	1	1
1	1	0	0	0	1	1	1
0	1	0	0	0	1	1	0
1	0	1	1	1	1	1	1

$*$

1	0
0	1

More Dimensions



$$x(u, v)$$

Question: How many input dimensions for x?

Answer: 2: u, v

Question: How many output dimensions for x?

Answer: 3 – red, green, and blue channels

$$x : \underbrace{\{0...255\}}_{\text{width}} \times \underbrace{\{0...255\}}_{\text{height}} \mapsto \underbrace{[0,1]^3}_{\text{Color values}}$$

More Dimensions



Computers represent 3 color channel each with 256 integer values

But we usually convert the colors $x(u, v) \in [0, 1]$ for scale reasons

$$\left[\frac{R}{255} \quad \frac{G}{255} \quad \frac{B}{255}\right]$$

More Dimensions

The pixels extend in 2 directions (height, width) $x(u, v)$

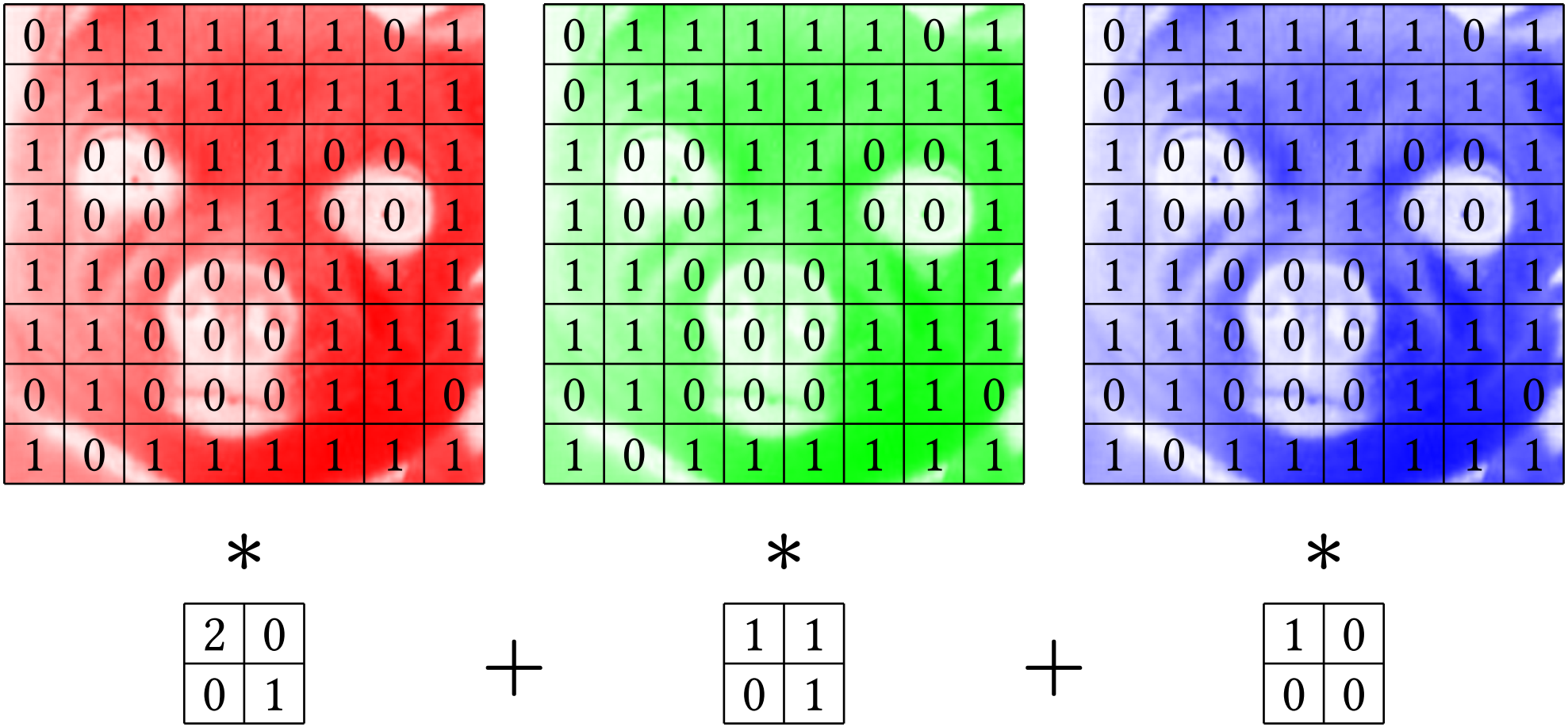
Each pixel contains 3 colors (channels) $x(u, v) = [r \;\; g \;\; b]^\top$

We can store this signal as a tensor

$$\boldsymbol{x} = \left[\underbrace{\begin{bmatrix} .13 & .14 & .12 & .10 \\ .80 & .40 & .20 & .05 \\ .15 & .08 & .75 & .16 \\ .21 & .38 & .90 & .78 \end{bmatrix}}_{\text{red}} \underbrace{\begin{bmatrix} .15 & .08 & .75 & .16 \\ .80 & .40 & .20 & .05 \\ .21 & .38 & .90 & .78 \\ .13 & .14 & .12 & .10 \end{bmatrix}}_{\text{green}} \underbrace{\begin{bmatrix} .80 & .40 & .20 & .05 \\ .21 & .38 & .90 & .78 \\ .15 & .08 & .75 & .16 \\ .13 & .14 & .12 & .10 \end{bmatrix}}_{\text{blue}} \right]^\top$$

This form is called CHW (channel, height, width) format

More Dimensions



More Dimensions

The multi-dimension, multi-channel, multi-filter equations are very long, let's skip them

Question: What is the shape of $\boldsymbol{\theta}$ for a single layer?

Answer:

$$\boldsymbol{\theta} \in \mathbb{R}^{c_x \times c_y \times k \times k + c_y}$$

- Input channels: c_x
- Output channels: $c_y + 1$
- Filter u (height): k
- Filter v (width): k

Convolve c_y filters of size $k \times k$ across c_x channels, bias for each output

More Dimensions

One last thing, **stride** allows you to "skip" cells during convolution

This can decrease the size of image without pooling

Padding adds zero pixels to the image to increase the output size

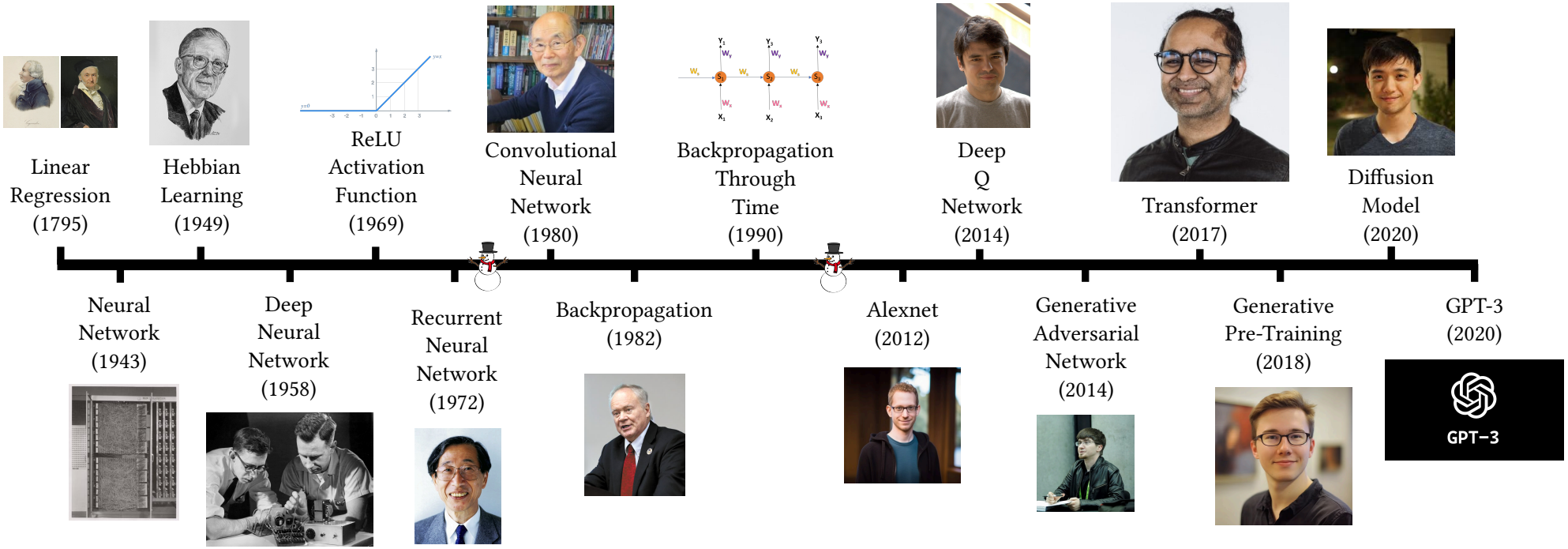
0	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1
1	0	0	1	1	0	0	1
1	0	0	1	1	0	0	1
1	1	0	0	0	1	1	1
1	1	0	0	0	1	1	1
0	1	0	0	0	1	1	0
1	0	1	1	1	1	1	1

$*$

1	0
0	1

Deeper Networks

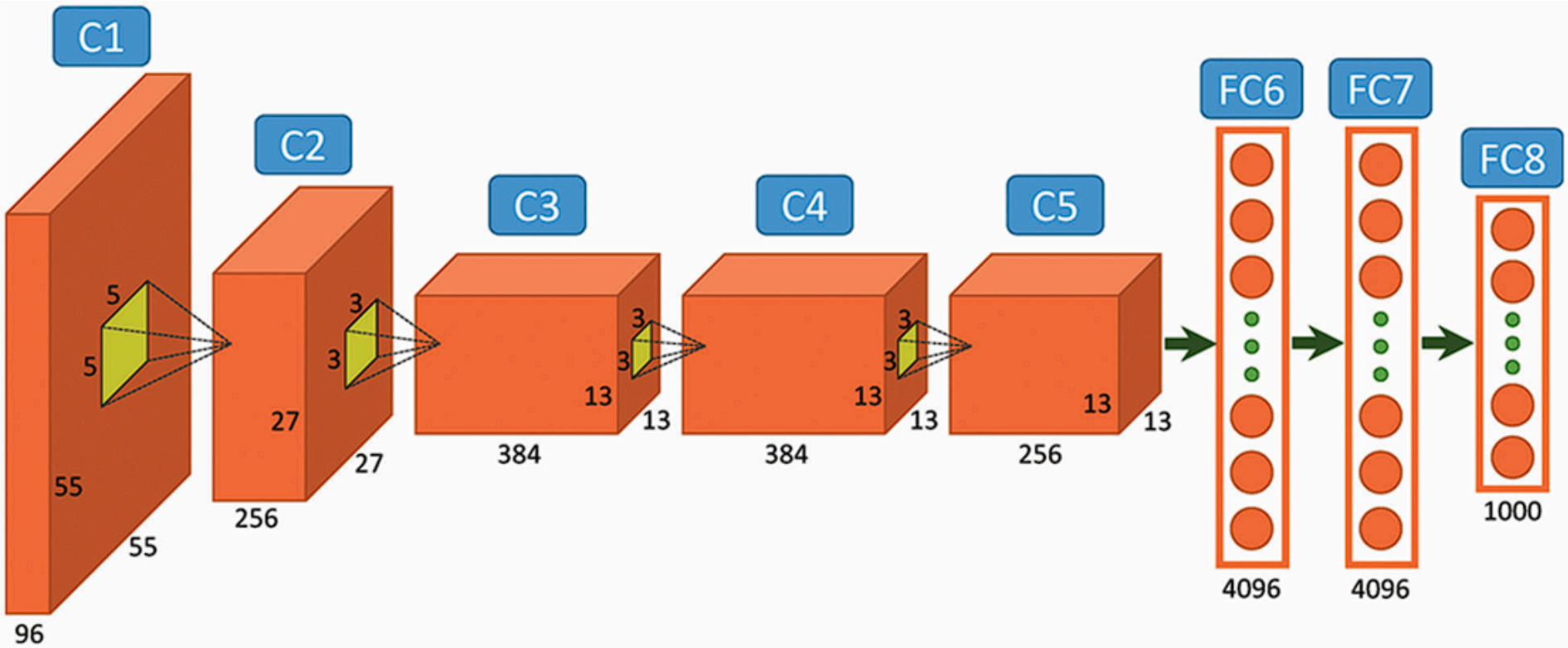
Deeper Networks



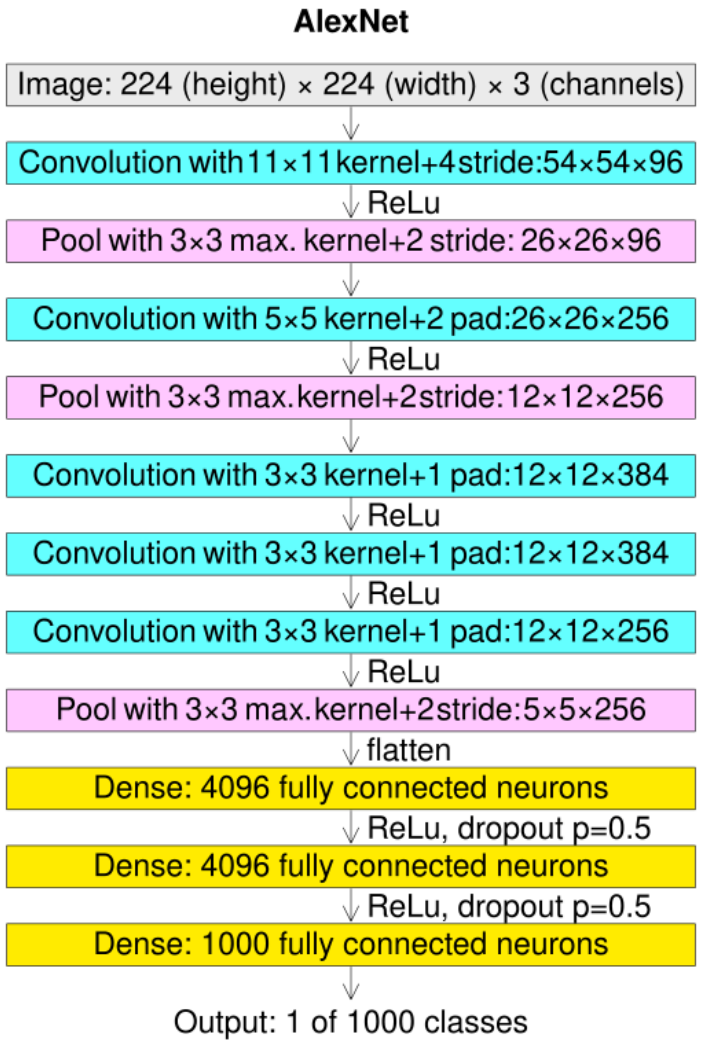
AlexNet: Train a neural network on a GPU ($n = 1.2m$, 6 days)

Paper by Krizhevsky, Sutskever (OpenAI CSO), and Hinton (Nobel)

Deeper Networks



Deeper Networks



Deeper Networks

Since AlexNet, there have been larger and better models

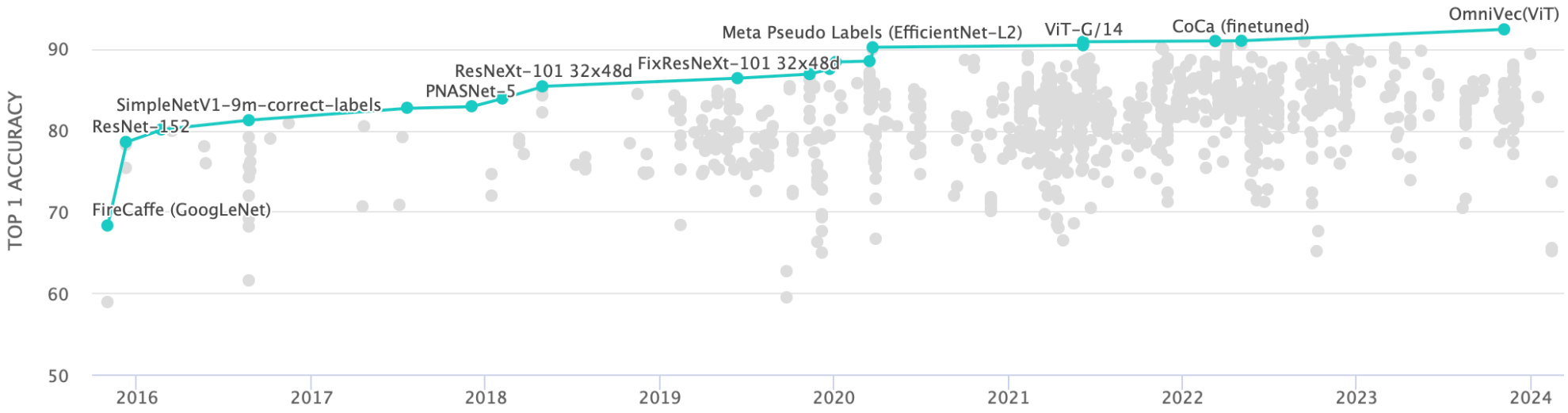
- VGG
- ResNet
- DenseNet
- MobileNet
- ResNeXt

ResNet, MobileNet, and ResNeXt are still used today!

Each one introduces a few tricks to obtain better results

Deeper Networks

After ResNet, marginal improvements



Coding

Coding

```
import torch
c_x = 3 # Number of colors
c_y = 32
k = 2 # Filter size
h, w = 128, 128 # Image size

conv1 = torch.nn.Conv2d(
  in_channels=c_x,
  out_channels=c_y,
  kernel_size=2
)
image = torch.rand((1, c_x, h, w)) # Torch requires BCHW
out = conv1(image) # Shape(1, c_y, h - k, w - k)
```

Coding

```
import torch
conv1 = torch.nn.Conv2d(3, c_h, 2)
pool1 = torch.nn.AdaptiveAvgPool2d((a, a))
conv2 = torch.nn.Conv2d(c_h, c_y, 2)
pool2 = torch.nn.AdaptiveAvgPool2d((b, b))
linear = torch.nn.Linear(c_y * b * b)
z_1 = conv1(image)
z_1 = torch.nn.functional.leaky_relu(z_1)
z_1 = pool1(z_1) # Shape(1, c_h, a, a)
z_2 = conv1(z1)
z_2 = torch.nn.functional.leaky_relu(z_2)
z_2 = pool2(z_2) # Shape(1, c_y, b, b)
z_3 = linear(z_2.flatten())
```

Coding

```
import jax, equinox
conv1 = equinox.nn.Conv2d(3, c_h, 2)
pool1 = equinox.nn.AdaptiveAvgPool2d((a, a))
conv2 = equinox.nn.Conv2d(c_h, c_y, 2)
pool2 = equinox.nn.AdaptiveAvgPool2d((b, b))
linear = equinox.nn.Linear(c_y * b * b)
z_1 = conv1(image,(3, h, w))
z_1 = jax.nn.leaky_relu(z_1)
z_1 = pool1(z_1) # Shape(c_h, a, a)
z_2 = conv1(z1)
z_2 = jax.nn.leaky_relu(z_2)
z_2 = pool2(z_2) # Shape(c_y, b, b)
z_3 = linear(z_2.flatten())
```

Coding

https://colab.research.google.com/drive/19rQJTU_m_q9ULeW1eq2i19JNaB-nrLWx?usp=sharing